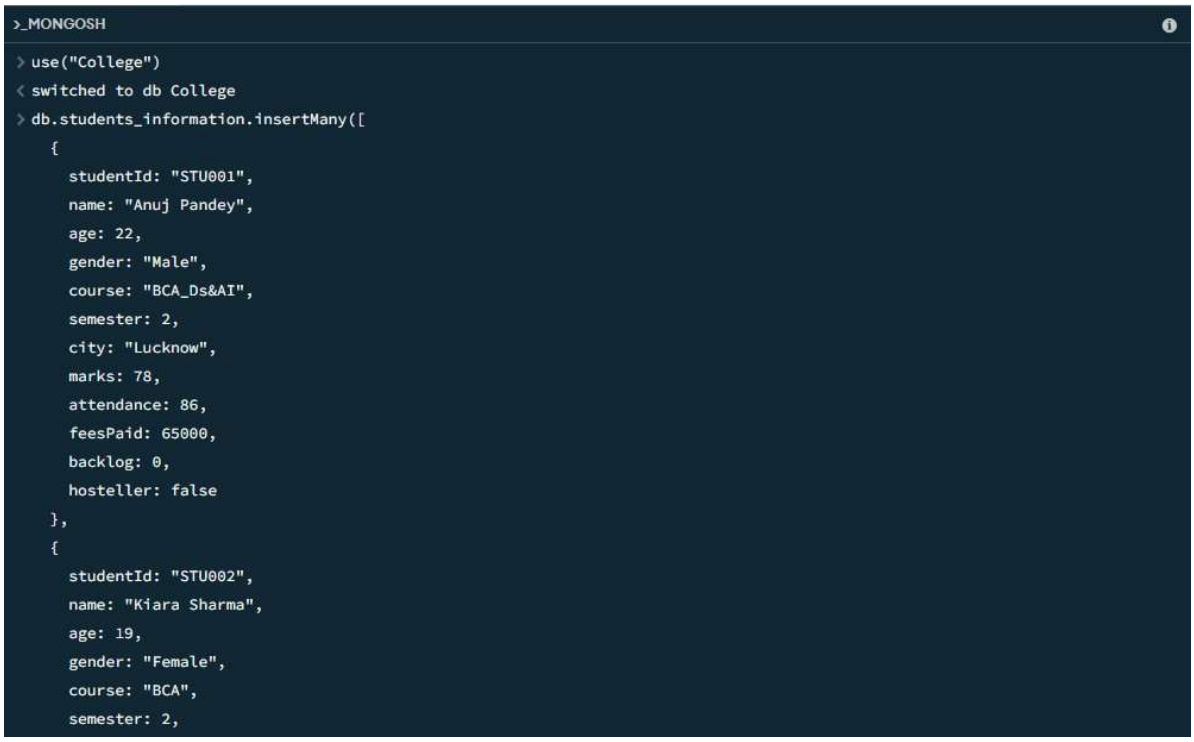
INSERTION OF STUDENT DOCUMENTS



```
>_MONGOSH
> use("College")
< switched to db College
> db.students_information.insertMany([
  {
    studentId: "STU001",
    name: "Anuj Pandey",
    age: 22,
    gender: "Male",
    course: "BCA_Ds&AI",
    semester: 2,
    city: "Lucknow",
    marks: 78,
    attendance: 86,
    feesPaid: 65000,
    backlog: 0,
    hosteller: false
  },
  {
    studentId: "STU002",
    name: "Kiara Sharma",
    age: 19,
    gender: "Female",
    course: "BCA",
    semester: 2,
```

Explanation: The supplied screenshot shows the MongoDB **insertMany()** command being used to insert student documents into the **students_information** collection.

6. CRUD / BASIC OPERATIONS — CREATE AND READ

CREATE — insertMany()

The Create part of CRUD is represented by inserting student documents into the collection using **insertMany()**.

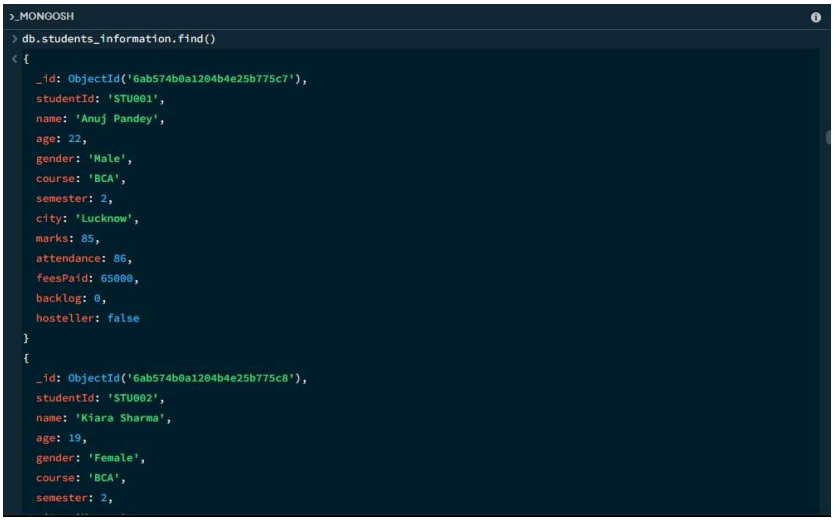


Figure: Supplied student documents returned from the collection.

READ — find()

The **find()** operation retrieves documents from the collection. The supplied screenshots show multiple student documents returned from students_information.



7. CRUD / BASIC OPERATIONS — READ ONE AND UPDATE

READ ONE — findOne()

The **findOne()** operation retrieves one document matching a specified condition. The supplied screenshot searches for **studentId: 'STU001'**.

```
>_MONGOSH
> db.students_information.findOne({
    studentId: "STU001"
  })
{
  _id: ObjectId('6ab574b0a1204b4e25b775c7'),
  studentId: 'STU001',
  name: 'Anuj Pandey',
  age: 22,
  gender: 'Male',
  course: 'BCA',
  semester: 2,
  city: 'Lucknow',
  marks: 85,
  attendance: 86,
  feesPaid: 65000,
  backlog: 0,
  hosteller: false
}
```

UPDATE — updateOne()

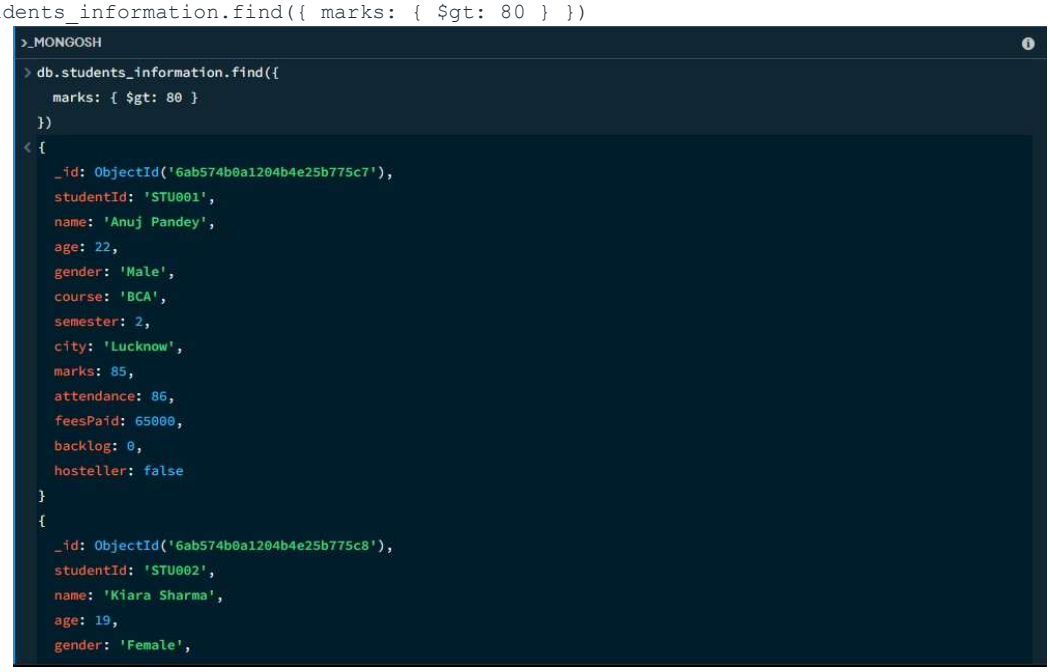
The **updateOne()** operation changes a selected field of a matching document. The supplied screenshot targets STU001 and uses the **$set** operator.

```
>_MONGOSH
> db.students_information.updateOne(
    { studentId: "STU001" },
    { $set: { marks: 85 } }
  )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

Evidence note: A deleteOne/deleteMany screenshot is not present in the supplied screenshots, so no delete command/result has been invented.

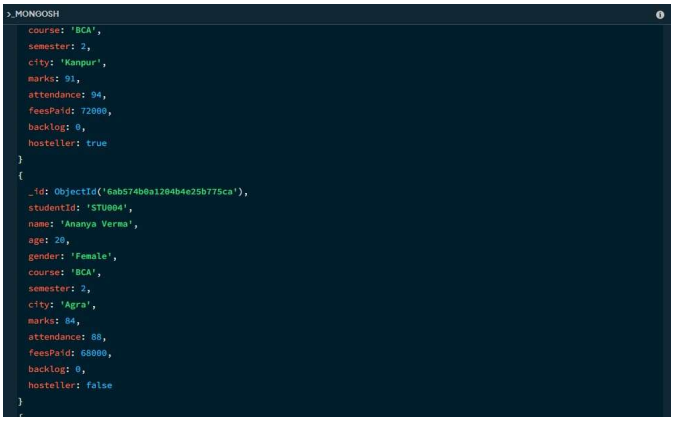
8. GREATER THAN — $gt

```
db.students_information.find({ marks: { $gt: 80 } })
```



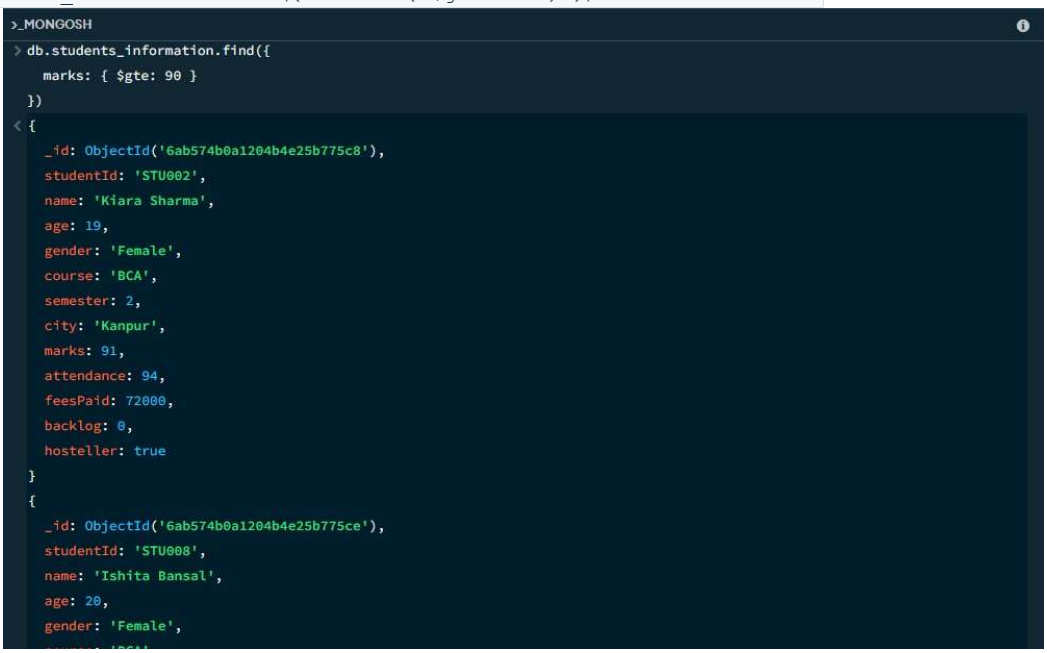
Explanation: The query filters students whose marks are greater than 80.

Continuation of the supplied result:



9. GREATER THAN OR EQUAL TO — $gte

```
db.students_information.find({ marks: { $gte: 90 } })
```



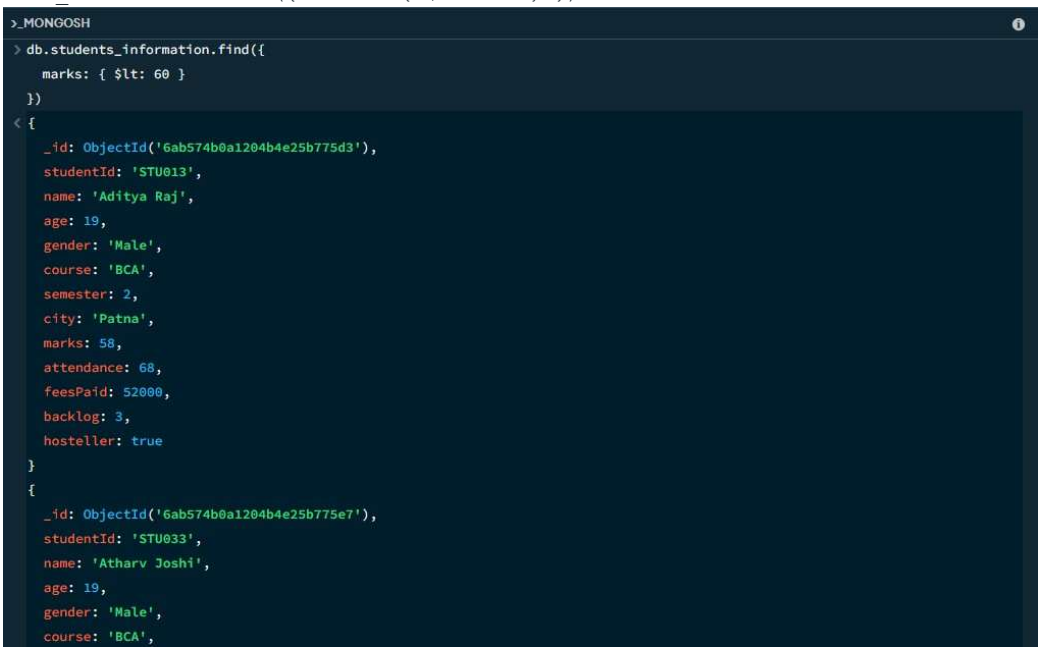


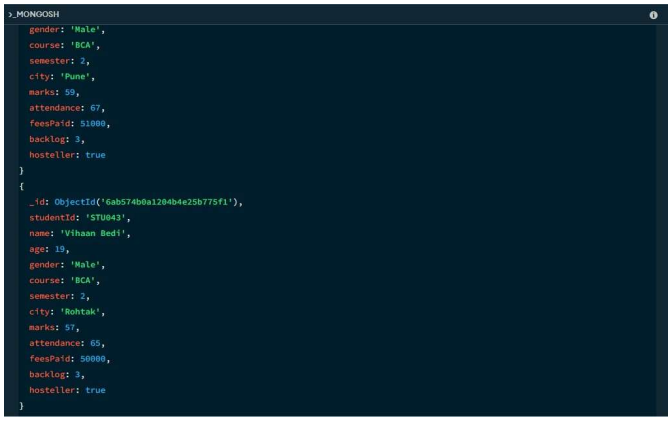
Explanation: The query filters students whose marks are greater than or equal to 90.

Continuation of the supplied result:

10. LESS THAN — $lt

```
db.students_information.find({ marks: { $lt: 60 } })
```





Explanation: The query filters students whose marks are less than 60.

Continuation of the supplied result:

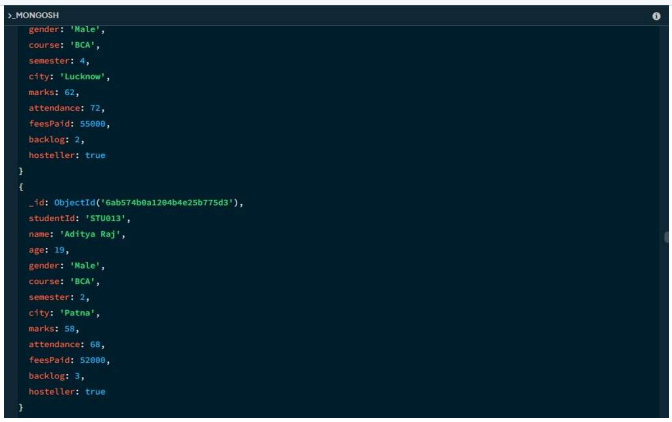
11. LESS THAN OR EQUAL TO — $lte

```
db.students_information.find({ marks: { $lte: 70 } })
```

```
>_MONGOSH
> db.students_information.find({
    marks: { $lte: 70 }
  })
< {
    _id: ObjectId('6ab574b0a1204b4e25b775c9'),
    studentId: 'STU003',
    name: 'Ritvik Malhotra',
    age: 21,
    gender: 'Male',
    course: 'BCA',
    semester: 4,
    city: 'Delhi',
    marks: 67,
    attendance: 79,
    feesPaid: 58000,
    backlog: 1,
    hosteller: true
  }
  {
    _id: ObjectId('6ab574b0a1204b4e25b775cd'),
    studentId: 'STU007',
    name: 'Kabir Saxena',
    age: 21,
    gender: 'Male',
    course: 'BCA',
```

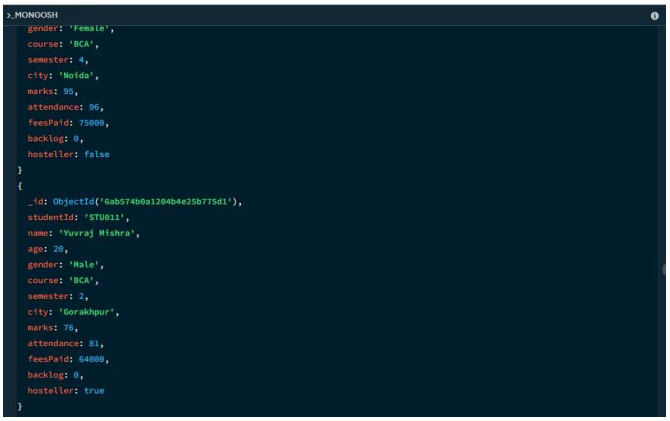
Explanation: The query filters students whose marks are less than or equal to 70.

Continuation of the supplied result:



12. EQUAL TO — $eq

```
db.students_information.find({ age: { $eq: 20 } })
```



```
>_MONGOSH
> db.students_information.find({
    age: { $eq: 20 }
  })
< {
    _id: ObjectId('6ab574b0a1204b4e25b775ca'),
    studentId: 'STU004',
    name: 'Ananya Verma',
    age: 20,
    gender: 'Female',
    course: 'BCA',
    semester: 2,
    city: 'Agra',
    marks: 84,
    attendance: 88,
    feesPaid: 68000,
    backlog: 0,
    hosteller: false
  }
  {
    _id: ObjectId('6ab574b0a1204b4e25b775ce'),
    studentId: 'STU008',
    name: 'Ishita Bansal',
    age: 20,
    gender: 'Female',
    course: 'BCA',
```

Explanation: The supplied screenshot uses an equality comparison on age to find students whose age is exactly 20.

Continuation of the supplied result:

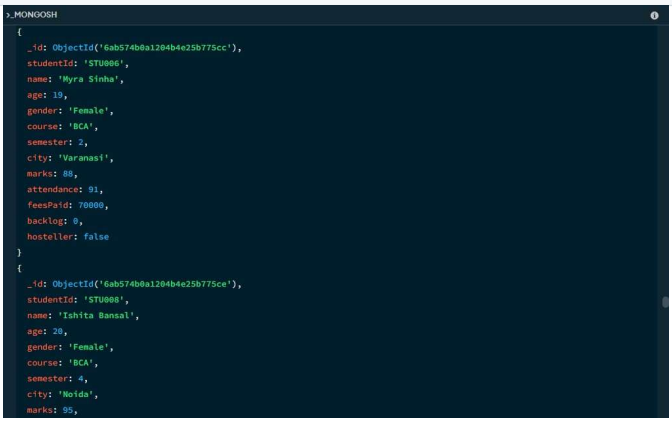
13. AND OPERATION — $and

```
db.students_information.find({ $and: [ { marks: { $gt: 80 } }, { attendance: { $gt: 90 } } ]
})
```



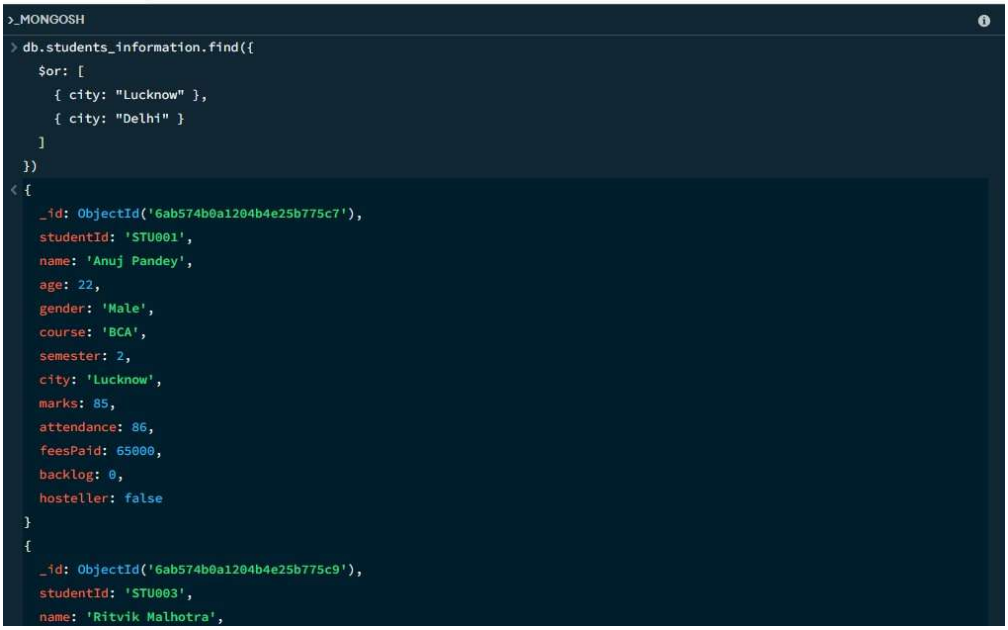
Explanation: The $and operator requires all specified conditions to be true. The supplied query combines marks and attendance conditions.

Continuation of the supplied result:



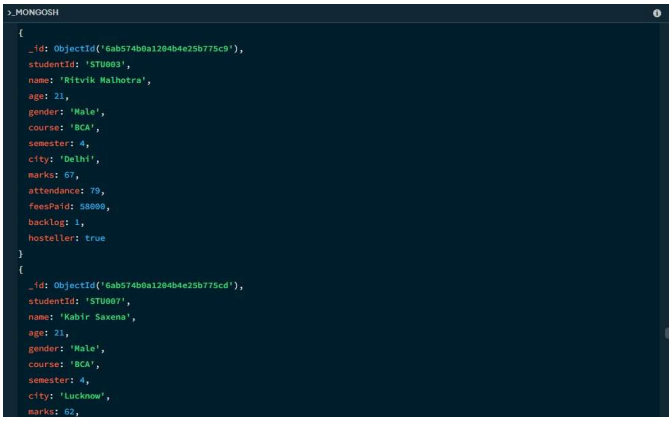
14. OR OPERATION — $or

```
db.students_information.find({ $or: [ { city: "Lucknow" }, { city: "Delhi" } ] })
```



Explanation: The $or operator returns documents satisfying at least one listed condition. The supplied query searches for students from Lucknow or Delhi.

Continuation of the supplied result:



15. NOT OPERATION — $not

```
db.students_information.find({ marks: { $not: { $gt: 80 } } })
```

```
>_MONGOSH
> db.students_information.find({
    marks: { $not: { $gt: 80 } }
  })
< {
    _id: ObjectId('6ab574b0a1204b4e25b775c9'),
    studentId: 'STU003',
    name: 'Ritvik Malhotra',
    age: 21,
    gender: 'Male',
    course: 'BCA',
    semester: 4,
    city: 'Delhi',
    marks: 67,
    attendance: 79,
    feesPaid: 58000,
    backlog: 1,
    hosteller: true
  }
  {
    _id: ObjectId('6ab574b0a1204b4e25b775cb'),
    studentId: 'STU005',
    name: 'Devansh Kapoor',
    age: 22,
    gender: 'Male',
    course: 'BCA',
```

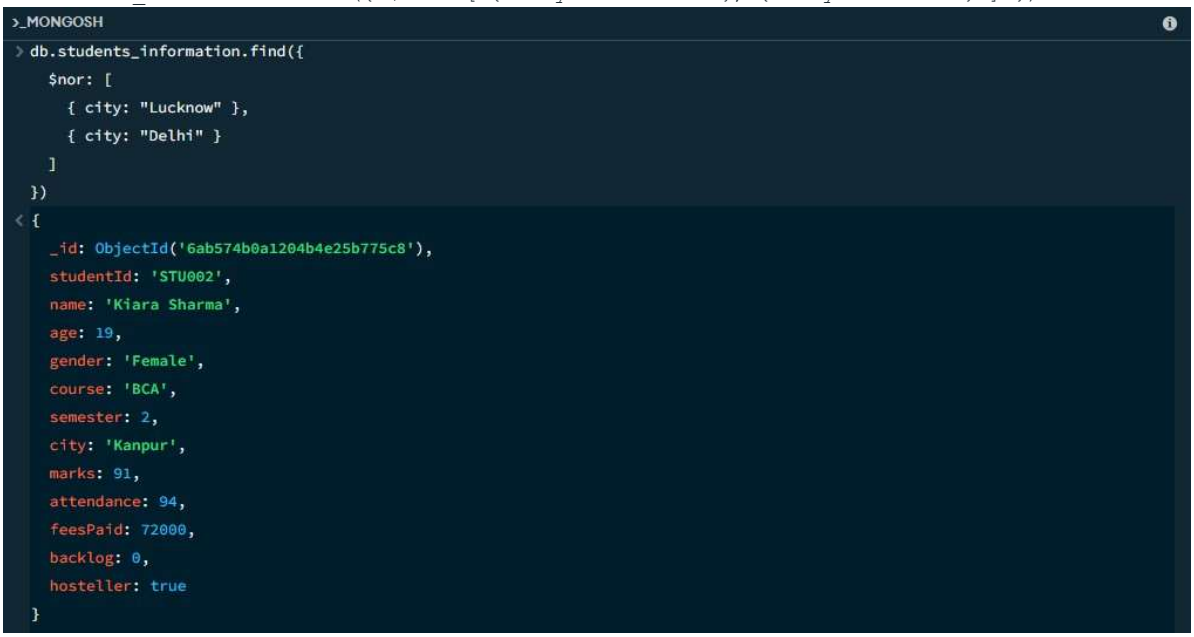
Explanation: The $not operator negates a condition. The supplied screenshot excludes marks greater than 80.

Continuation of the supplied result:

```
>_MONGOSH
  {
    _id: ObjectId('6ab574b0a1204b4e25b775cb'),
    studentId: 'STU005',
    name: 'Devansh Kapoor',
    age: 22,
    gender: 'Male',
    course: 'BCA',
    semester: 6,
    city: 'Jaipur',
    marks: 73,
    attendance: 75,
    feesPaid: 61000,
    backlog: 2,
    hosteller: true
  }
  {
    _id: ObjectId('6ab574b0a1204b4e25b775cd'),
    studentId: 'STU007',
    name: 'Kabir Saxena',
    age: 21,
    gender: 'Male',
    course: 'BCA',
    semester: 4,
    city: 'Lucknow',
    marks: 62,
```

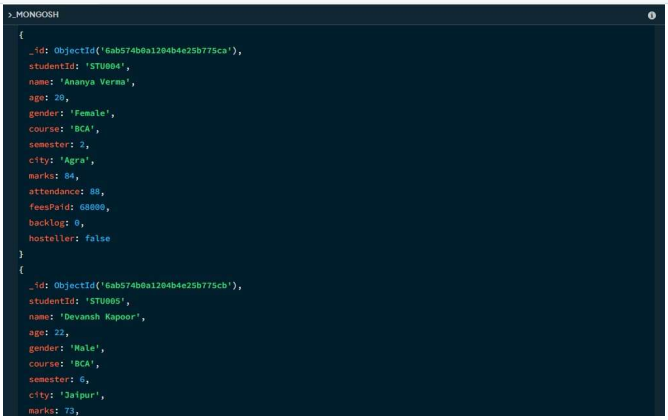
16. NOR OPERATION — $nor

```
db.students_information.find({ $nor: [ { city: "Lucknow" }, { city: "Delhi" } ] })
```



Explanation: The $nor operator returns documents that do not satisfy any of the listed conditions. The supplied query uses city conditions.

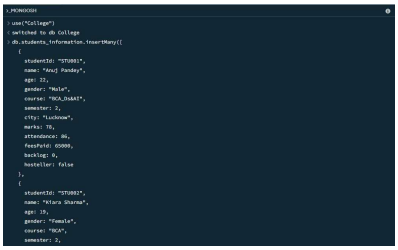
Continuation of the supplied result:


>_MONGOSH
{
_id: ObjectId('6ab574b0a1204b4e25b775ca'),
studentId: 'STU004',
name: 'Ananya Verma',
age: 20,
gender: 'Female',
course: 'BCA',
semester: 2,
city: 'Agra',
marks: 84,
attendance: 88,
feesPaid: 68000,
backlog: 0,
hosteller: false
}
{
_id: ObjectId('6ab574b0a1204b4e25b775cb'),
studentId: 'STU005',
name: 'Devansh Kapoor',
age: 22,
gender: 'Male',
course: 'BCA',
semester: 6,
city: 'Jaipur',
marks: 73,

21. APPENDIX — SUPPLIED SCREENSHOTS 1–6

The screenshots below are the original MongoDB practical evidence supplied for this project.

Screenshot 12147



Screenshot 12452



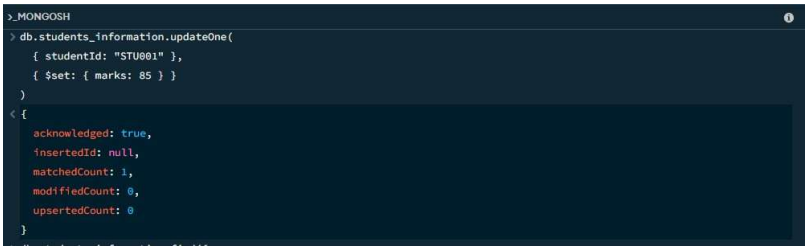
Screenshot 12630



Screenshot 12722



Screenshot 13351



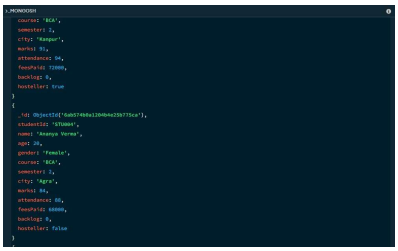
Screenshot 13456


MONGOSH
db.students_information.find({
marks: { $gt: 80 }
})
{
_id: ObjectId('6a8d748b6a1204b4e25b775c7'),
studentId: 'STU0063',
name: 'Anuj Pandey',
age: 22,
gender: 'Male',
course: 'BCA',
semester: 3,
city: 'Lucknow',
mark1: 88,
attendance: 89,
feesPaid: 65000,
backlog: 0,
hostelier: false
}
{
_id: ObjectId('6a8d748b6a1204b4e25b775c8'),
studentId: 'STU0063',
name: 'Kiara Sharma',
age: 19,
gender: 'Female',

22. APPENDIX — SUPPLIED SCREENSHOTS 7–12

The screenshots below are the original MongoDB practical evidence supplied for this project.

Screenshot 13542



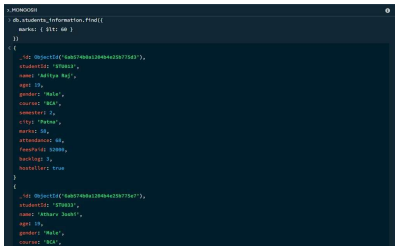
Screenshot 13623



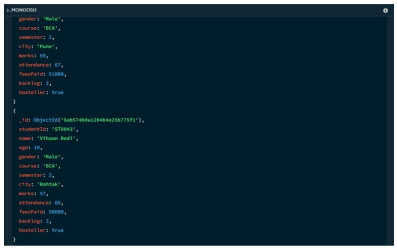
Screenshot 13734



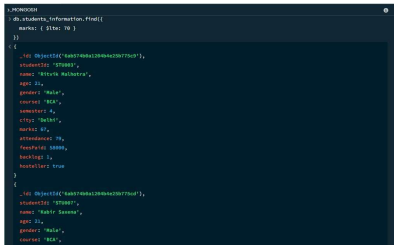
Screenshot 13903



Screenshot 13945



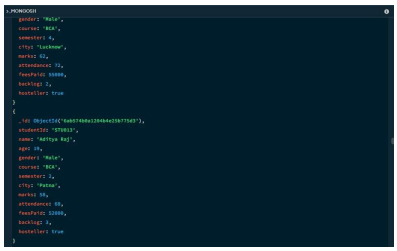
Screenshot 14023


MONGOSH
> db.students_information.find({
marks: { $lte: 70 }
})
{
_id: ObjectId('6a69749a6a1204b4e25977bc9'),
studentId: 'ST0003',
name: 'Ritik Malhotra',
age: 21,
gender: 'Male',
course: 'BCA',
semester: 4,
city: 'Delhi',
marks: 67,
attendance: 78,
feesPaid: 50000,
backlog: 1,
hostelier: true
}
{
_id: ObjectId('6a69749a6a1204b4e25977bcd'),
studentId: 'ST0007',
name: 'Rakhi Saxena',
age: 21,
gender: 'Male',
course: 'BCA',

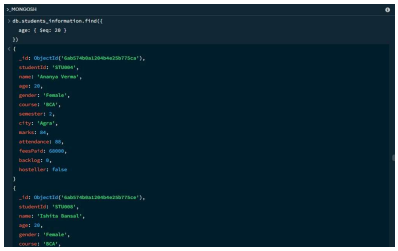
23. APPENDIX — SUPPLIED SCREENSHOTS 13–18

The screenshots below are the original MongoDB practical evidence supplied for this project.

Screenshot 14105



Screenshot 14214



Screenshot 14251



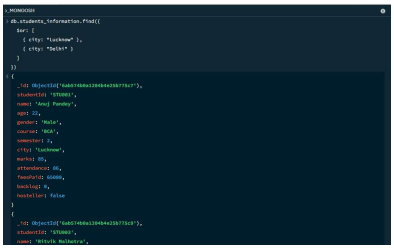
Screenshot 14336



Screenshot 14538



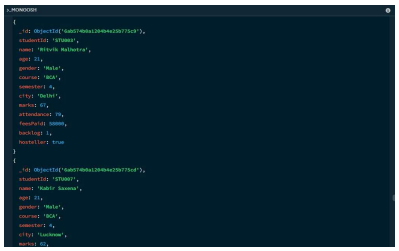
Screenshot 14627


db.students_information.find({
$or: [
{ city: "Lucknow" },
{ city: "Delhi" }
]
})
{
_id: ObjectId('6a867468e1304b4e21b773c7'),
studentId: 'STU001',
name: 'Anuj Pandey',
age: 22,
gender: 'Male',
course: 'BCA',
semester: 3,
city: 'Lucknow',
marks: 85,
attendance: 88,
feesPaid: 61000,
backlog: 0,
hostelier: false
}
{
_id: ObjectId('6a867468e1304b4e21b773c8'),
studentId: 'STU002',
name: 'Nitesh Malhotra',

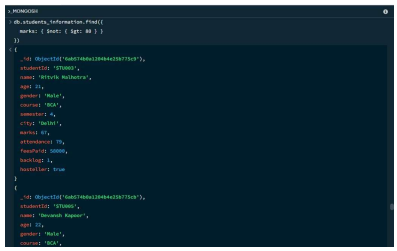
24. APPENDIX — SUPPLIED SCREENSHOTS 19–24

The screenshots below are the original MongoDB practical evidence supplied for this project.

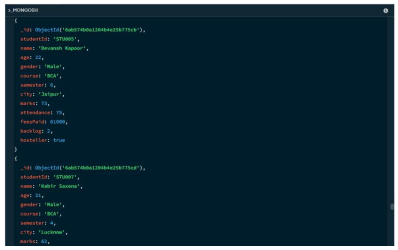
Screenshot 14701



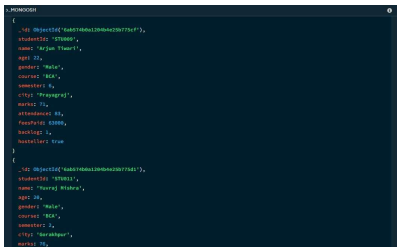
Screenshot 14745



Screenshot 14836



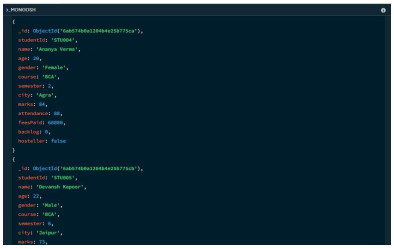
Screenshot 14912



Screenshot 14954



Screenshot 15046


MONGODB
{
_id: ObjectId('6a6674b8e3204b4e25b775ca'),
studentId: 'STU004',
name: 'Ananya Verma',
age: 20,
gender: 'Female',
course: 'BCA',
semester: 3,
city: 'Agra',
marks: 84,
attendance: 88,
feesPaid: 60000,
backlogs: 0,
hosteller: false
}
{
_id: ObjectId('6a6674b8e3204b4e25b775cb'),
studentId: 'STU005',
name: 'Devansh Kapoor',
age: 22,
gender: 'Male',
course: 'BCA',
semester: 6,
city: 'Jaipur',
marks: 72,